

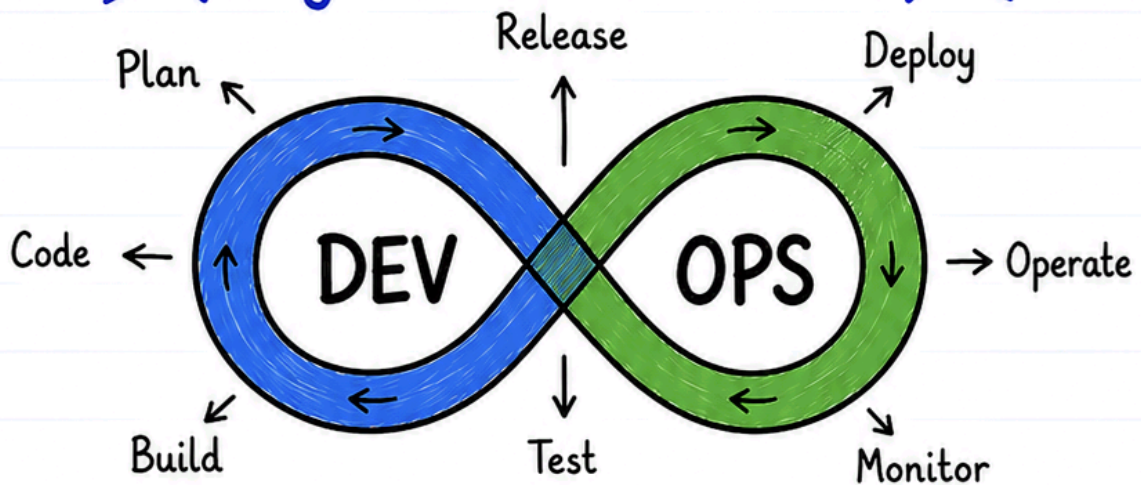


@qa_insights

DEVOPS

COMPLETE NOTES

≧ (Beginner to Advanced) ≦



Learn **DevOps** step-by-step
Bridge **Dev** & **Ops** like a **Pro**!



CI/CD



Docker



Kubernetes



Jenkins



Monitoring



Git



FOLLOW



SHARE



COMMENT



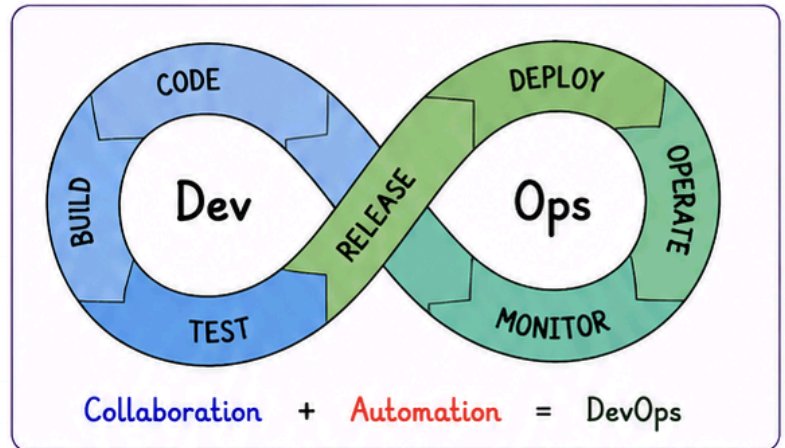


DEVOPS COMPLETE NOTES

Your complete **beginner-to-advanced** guide to learn **DevOps** from scratch!

★ What You'll Learn

- ✓ DevOps fundamentals and culture
- ✓ DevOps lifecycle & best practices
- ✓ CI/CD with Jenkins
- ✓ Version control with Git & GitHub
- ✓ Build tools with Maven
- ✓ Containerization with Docker
- ✓ Orchestration with Kubernetes
- ✓ Monitoring, Logging & more
- ✓ DevOps in Automation Testing



👥 Who Should Learn?

- ✓ QA Engineers & Test Automation Engineers
- ✓ Developers
- ✓ System Administrators
- ✓ Anyone who wants to build a career in DevOps

🎯 Why DevOps Matters?

- ✓ Faster delivery of **high-quality software**.
- ✓ **Better collaboration** between Dev & Ops teams.
- ✓ **Automation** reduces manual work and errors.
- ✓ Improved **reliability, scalability & security**.
- ✓ Continuous feedback and **faster releases**.

📖 DevOps Roadmap

- 1 Learn the Fundamentals → DevOps concepts, culture, mindset
- 2 Version Control → Git, GitHub
- 3 Build & Automate → Maven, Build tools
- 4 CI/CD Pipeline → Jenkins, Pipelines
- 5 Containerization → Docker, Docker Compose
- 6 Orchestration → Kubernetes basics
- 7 Monitoring & Logging → Prometheus, Grafana, ELK
- 8 DevOps in QA → Automation, Reporting, Integration
- 9 Real-world Projects → End-to-end DevOps implementation



From QA Perspective

As a QA, understanding DevOps helps you build **reliable automation**, integrate tests in **CI/CD pipelines**, and deliver quality software faster with **confidence**!



Test



Automate



Deliver Faster

💖 Keep Learning, Keep Building, Keep Growing! 🚀

★ Let's master **DevOps** together!





WHAT IS DEVOPS?

DevOps is a combination of Development (Dev) and Operations (Ops) practices, tools and culture to automate and integrate the process of software development and IT operations.

★ Definition

DevOps is a culture and set of practices that aims to shorten the systems development life cycle and provide continuous delivery with high software quality.



Goals of DevOps

- ✓ Deliver value to customers **faster**
- ✓ Improve **collaboration** between teams
- ✓ **Automate** manual processes
- ✓ Increase **efficiency** and **productivity**
- ✓ Ensure high **quality** & **reliability**



Benefits

- ✓ **Faster** time to market
- ✓ Continuous **delivery** & **deployment**
- ✓ **Reduced** failures and rollbacks
- ✓ Better **scalability** & **stability**
- ✓ Higher **customer satisfaction**
- ✓ Improved **feedback** & **visibility**



DevOps Culture

- ✓ **Collaboration** over Silos
- ✓ **Automation** over Manual Work
- ✓ Continuous **Improvement**
- ✓ **Transparency** over Hiding
- ✓ **Shared** Responsibility
- ✓ **Customer-Centric** Approach



Why Companies Use DevOps?

- ✓ To handle **dynamic market changes** quickly
- ✓ To **reduce** manual work and human errors
- ✓ To improve product **quality** and **reliability**
- ✓ To achieve business goals **faster**
- ✓ To gain **competitive advantage**



DevOps connects every step of the journey and makes it continuous!



Key Takeaway

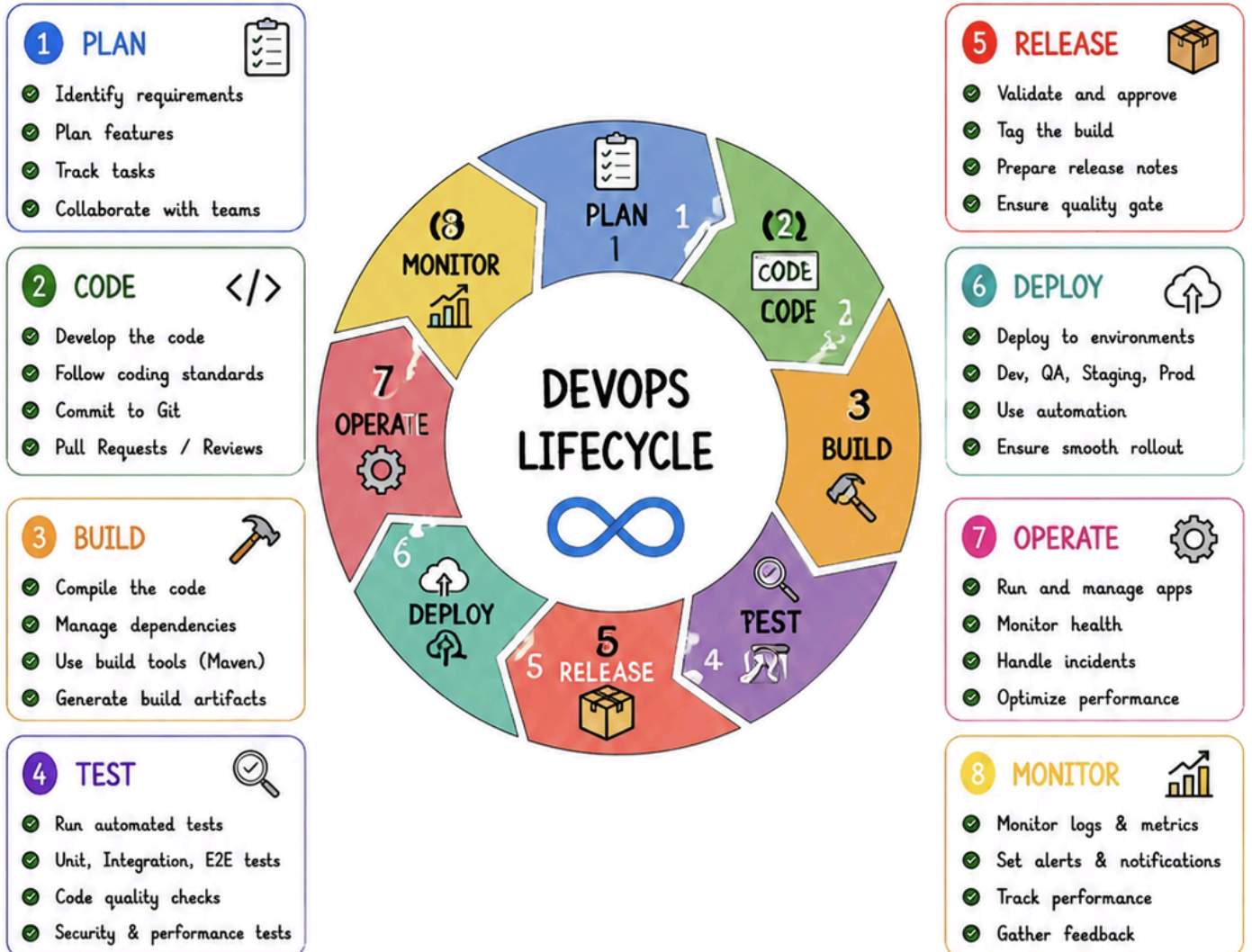
DevOps is not just about **tools**, it's about **people**, **process** and **culture** working together to deliver better software, faster and smarter.





DEVOPS LIFECYCLE

The DevOps Lifecycle is a **continuous cycle** that helps deliver high-quality software **faster** and more **reliably**.



Key Takeaway

DevOps is not a one-time process, it's a **never-ending cycle** of **improvement** and **collaboration**.



Continuous
Cycle



Better
Collaboration



Faster
Delivery



Business
Success



Continuous Integration + Continuous Delivery = Business Value

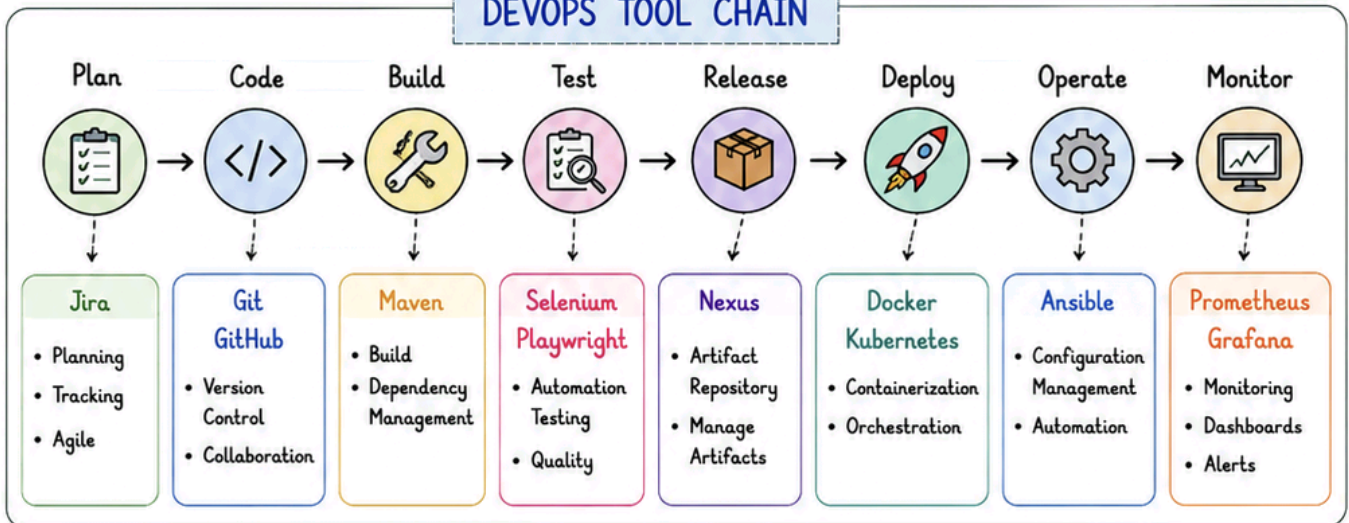




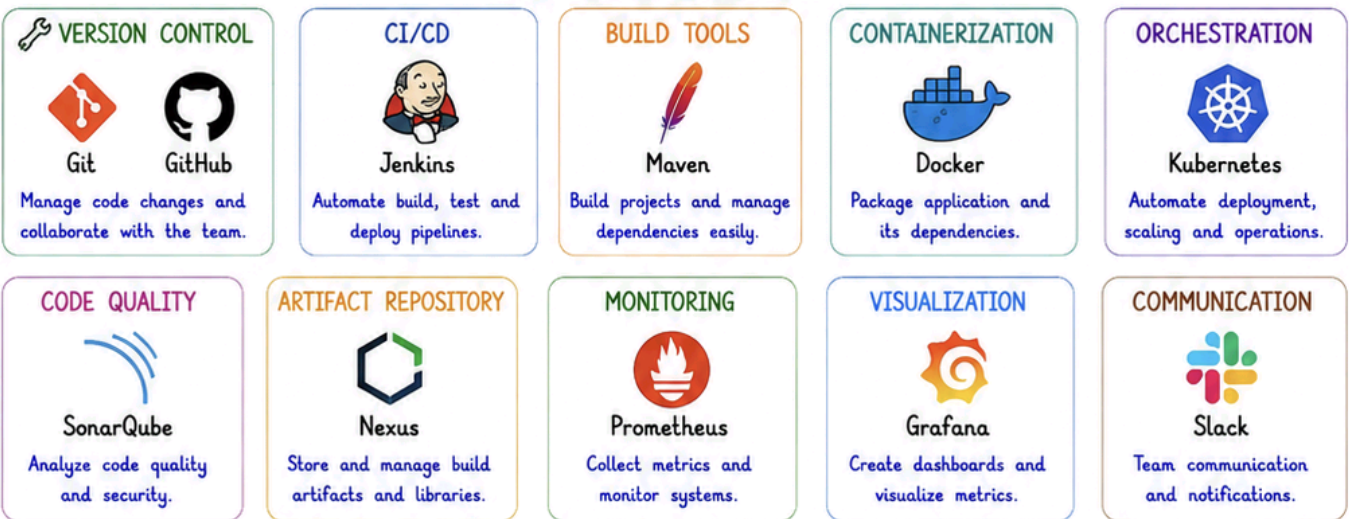
DEVOPS TOOLS ECOSYSTEM

DevOps uses a set of **powerful tools** that work together at different stages of the lifecycle to **automate, integrate and deliver** software faster.

DEVOPS TOOL CHAIN



TOOLS BY CATEGORY



Key Takeaway

These tools may look different, but they work together as a team to make DevOps **faster, smarter and reliable**.

Right Tool

Right Process

Right People



Successful DevOps



Remember



- ✓ Choose the right tool for the job.
- ✓ Integrate tools for maximum value.
- ✓ Automate everything you can.
- ✓ Monitor and improve continuously.

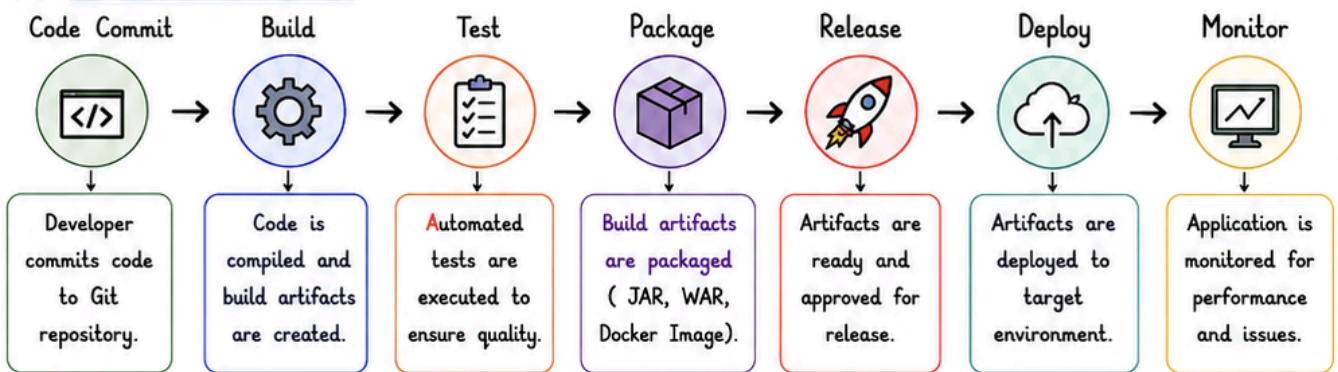


CI/CD EXPLAINED

CI/CD stands for **Continuous Integration**, **Continuous Delivery** and **Continuous Deployment**.

It is a DevOps practice that helps deliver **high-quality** software **faster** and **reliably**.

★ THE CI/CD PIPELINE



🎯 KEY POINTS

- ✓ **CI**: Developers integrate code frequently.
- ✓ **CD (Delivery)**: Code is always in a deployable state.
- ✓ **CD (Deployment)**: Code is automatically deployed to production.
- ✓ Automates build, test, release and deployment.
- ✓ Reduces manual work and human errors.
- ✓ Provides faster feedback to developers.
- ✓ Improves collaboration and quality.

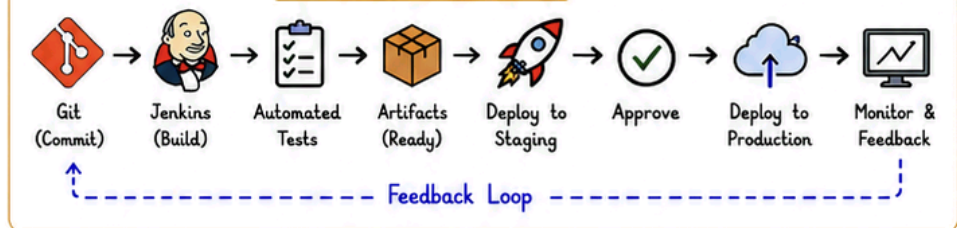
⚖️ CI vs CD vs CD (Deployment)

Aspect	CI	CD (Delivery)	CD (Deployment)
Meaning	Integrate & test code frequently	Deliver code to staging/production when ready	Automatically deploy code to production
Trigger	On every code commit	Manual approval	Automatic
Focus	Integration & Testing	Release Management	Automatic Deployment
Environment	Dev / Build Environment	Staging / UAT Environment	Production Environment
Human Intervention	Minimal	Required	Not Required
Goal	Find bugs early	Release anytime with confidence	Release as soon as possible

📁 BENEFITS OF CI/CD

- ✓ **Faster** delivery of features
- ✓ **Early** bug detection
- ✓ **High** code quality
- ✓ **Better** collaboration between teams
- ✓ **Consistent** and reliable releases
- ✓ **Improved** customer satisfaction

CI/CD FLOW EXAMPLE



</> EXAMPLE (JENKINS PIPELINE SNIPPET)

```

1 pipeline {
2   agent any
3   stages {
4     stage('Build') { steps { sh 'mvn clean package' } }
5     stage('Test') { steps { sh 'mvn test' } }
6     stage('Deploy') { steps { sh 'docker build -t myapp .' } }
7   }
8 }
  
```

This pipeline builds, tests and packages the application.

📌 REMEMBER

- ✓ CI is about **Integration & Testing**.
- ✓ CD (Delivery) is about **Releasing** safely.
- ✓ CD (Deployment) is about **Deploying** automatically.
- ✓ CI/CD = **Speed** + **Quality** + **Reliability**
- ✓ Automate **Everything!**



GIT IN DEVOPS

Git is a **Distributed Version Control System** that helps teams **track changes**, **collaborate** and **manage code** efficiently.

★ WHAT IS GIT?

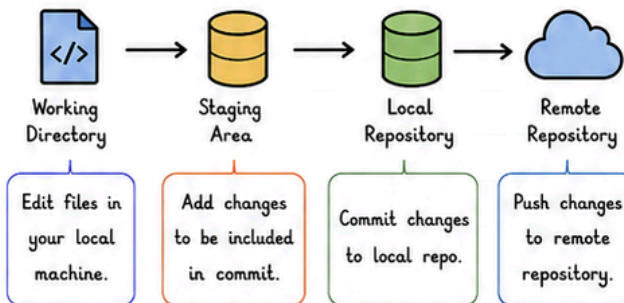
- ✔ Tracks every change in the code.
- ✔ Allows multiple developers to work together.
- ✔ Stores code in a remote repository.
- ✔ Enables branching, merging and collaboration.
- ✔ Essential for CI/CD pipelines.



★ WHY GIT IN DEVOPS?

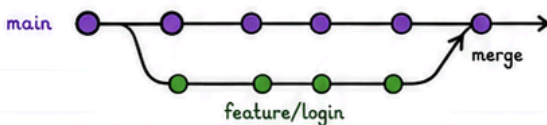
- ✔ **Version Control** - Track every change.
- ✔ **Collaboration** - Multiple people can work together.
- ✔ **Branching** - Work on features independently.
- ✔ **Integration** - Works with CI/CD tools like Jenkins.
- ✔ **Backup** - Remote repositories act as backup.

★ GIT WORKFLOW



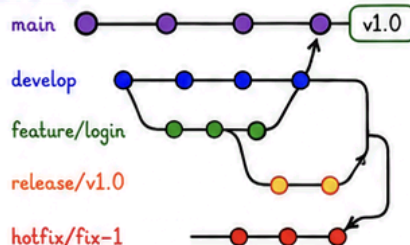
★ BRANCHING IN GIT

- ✔ Branching allows you to work on features independently.
- ✔ After testing, merge the branch into main branch.
- ✔ Keeps the main code stable.



★ COMMON GIT BRANCHING STRATEGY

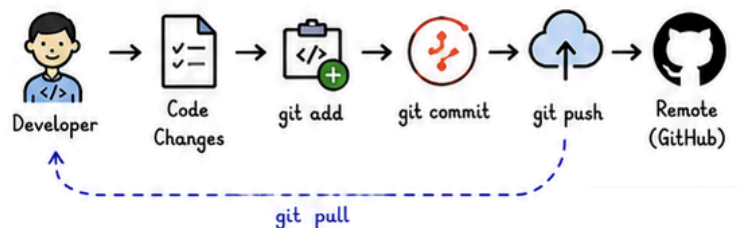
- main / master** - Stable production code.
- develop** - Integration branch for features.
- feature/*** - New features or changes.
- release/*** - Release preparation.
- hotfix/*** - Fix for production issues.



★ GIT BASIC COMMANDS

<code>git init</code>	# Initialize a new repository
<code>git clone <url></code>	# Clone a remote repository
<code>git add <file></code>	# Add file to staging area
<code>git commit -m "msg"</code>	# Commit changes
<code>git status</code>	# Check status of changes
<code>git push origin <branch></code>	# Push to remote repo
<code>git pull origin <branch></code>	# Pull from remote repo
<code>git branch</code>	# List all branches
<code>git checkout <branch></code>	# Switch to a branch
<code>git merge <branch></code>	# Merge a branch

★ GIT WORKFLOW EXAMPLE



★ GIT BEST PRACTICES

- ✔ Write meaningful commit messages.
- ✔ Pull latest changes before pushing.
- ✔ Create feature branches for new work.
- ✔ Do code reviews using Pull Requests.
- ✔ Keep commits small and atomic.
- ✔ Never push sensitive data.



KEY TAKEAWAY

Git helps teams **collaborate** efficiently, manage **code versions** and integrate smoothly with DevOps tools for **automation** and **deployment**.





JENKINS

Jenkins is an **open-source** automation server used to **build, test and deploy** applications continuously. It is a key tool in **CI/CD pipelines**.

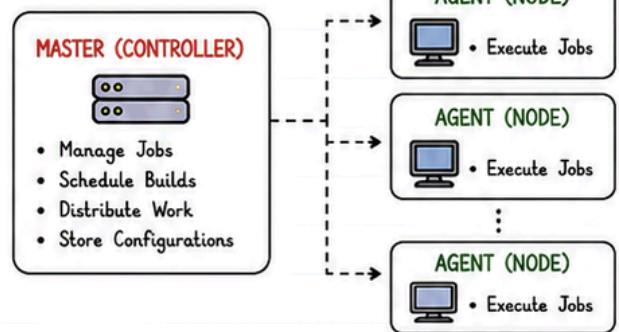
★ 1. WHAT IS JENKINS?

- Open-source automation server written in **Java**.
- Helps automate the **build, test** and **deployment** process.
- Supports **1000+** plugins.
- Extensible and easy to configure.
- Works on **Master-Slave (Controller-Agent)** architecture.
- Widely used for CI/CD in DevOps.



Jenkins

★ 2. JENKINS ARCHITECTURE



Master assigns jobs to Agents (Nodes) to execute.

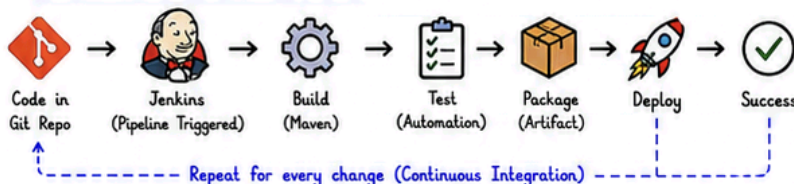
★ 3. JENKINS COMPONENTS

Job	Build	Plugin	Workspace	Credentials
Defines a task to be executed by Jenkins.	An instance of job execution with a unique number.	Extends Jenkins functionality (1000+ plugins).	Directory where the job is executed.	Stores usernames, passwords, tokens securely.

★ 4. JENKINS JOB TYPES

- Freestyle Project** - Simple job, build steps defined manually.
- Pipeline** - Define entire CI/CD as code (Jenkinsfile).
- Multi-configuration** - Run the job with multiple configurations.
- Folder** - Organize multiple jobs.
- Multibranch Pipeline** - Create pipeline for each branch.

★ 5. JENKINS PIPELINE FLOW



★ 6. BUILD STATUS

- SUCCESS** - Build completed successfully.
- UNSTABLE** - Some tests failed.
- FAILURE** - Build failed.
- ABORTED** - Build was stopped.

★ 7. KEY FEATURES

- Easy installation and configuration.
- Huge plugin ecosystem.
- Distributed builds using Agents.
- Pipeline as Code (**Jenkinsfile**).
- Integrates with many DevOps tools.
- Email / Slack notifications.

POPULAR PLUGINS

- Git Plugin** - Integrate with Git.
- Pipeline Plugin** - CI/CD Pipelines.
- JUnit Plugin** - Test reports.
- Maven Plugin** - Build with Maven.
- Slack Plugin** - Notifications.
- Docker Plugin** - Build & Deploy images.

★ 8. BASIC JENKINS STEPS

1. Install Jenkins.
2. Create a new job.
3. Configure source code (Git).
4. Add build steps (e.g., Maven).
5. Add post-build actions (e.g., publish reports, deploy).
6. Save and Build Now.



KEY TAKEAWAY

Jenkins **automates** the build, test and deployment process, making **CI/CD** faster, reliable and consistent.





MAVEN

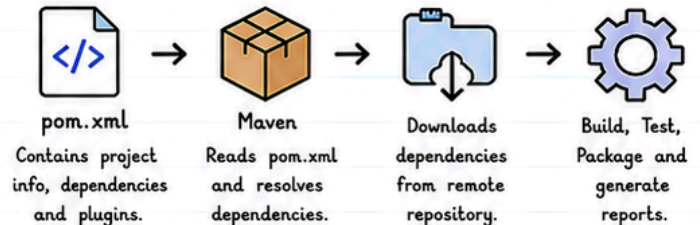
Maven is a **Build Automation** tool used for **Java** projects.
It simplifies the build process, manages **dependencies** and standardizes project structure.

★ 1. WHAT IS MAVEN?

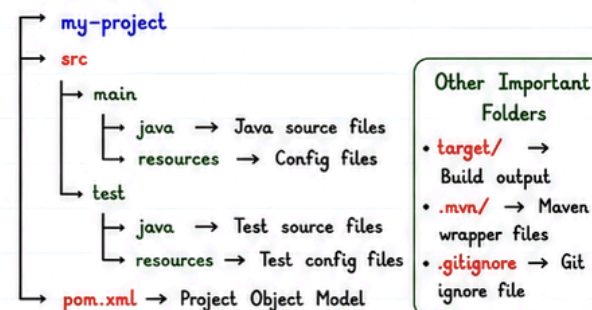
- ✓ Apache Maven is an **open-source** build automation tool.
- ✓ It uses a **POM** (Project Object Model) to manage project build, dependencies, plugins and reporting.
- ✓ Follows **convention** over configuration.
- ✓ Widely used in **CI/CD pipelines**.



★ 2. HOW MAVEN WORKS?



★ 3. MAVEN PROJECT STRUCTURE



★ 4. POM.XML (PROJECT OBJECT MODEL)

POM is the **heart** of Maven project.
It contains project info, **dependencies**, plugins and configurations.

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.qa</groupId>
  <artifactId>my-project</artifactId>
  <version>1.0.0</version>
</project>
  
```

★ 5. DEPENDENCY MANAGEMENT

- ✓ Maven automatically downloads required libraries.
- ✓ Add dependencies in pom.xml.

```

<dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.20.0</version>
  </dependency>
</dependencies>
  
```



★ 6. MAVEN LIFECYCLE

Maven has **3** built-in lifecycles.

Lifecycle	Purpose	Phases (Example)
1. clean	Cleans the project	clean
2. default	Builds and deploys	compile, test, package, verify, install, deploy
3. site	Generates project docs	site

Typical Flow (Default Lifecycle)



★ 7. USEFUL MAVEN COMMANDS

Command	Purpose
mvn clean	Cleans the target directory
mvn compile	Compiles the source code
mvn test	Runs the tests
mvn package	Packages the project (JAR/WAR)
mvn install	Installs the package to local repo
mvn deploy	Deploys to remote repository
mvn -v	Shows Maven version

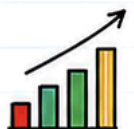
★ 8. FEATURES

- ✓ Automatic dependency management
- ✓ Standard project structure
- ✓ Built-in lifecycle and goals
- ✓ Extensible with plugins
- ✓ Cross-platform
- ✓ Easy integration with CI tools (Jenkins, GitLab CI, etc.)



★ 9. BENEFITS OF MAVEN

- ✓ Reduces build time
- ✓ Consistent builds across environments
- ✓ Easy dependency management
- ✓ Improved project maintainability
- ✓ Encourages best practices
- ✓ Seamless CI/CD integration



KEY TAKEAWAY

Maven **standardizes** the build process, manages **dependencies** and makes projects easier to **build**, **test** and **deploy**.





DOCKER

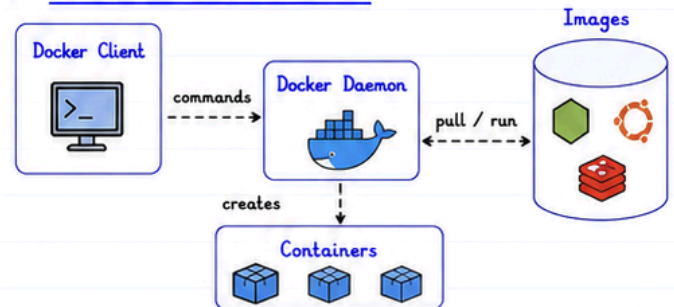
Docker is a platform that helps you **build**, **ship** and **run** applications inside **containers**.

★ 1. WHAT IS DOCKER?

- ✓ Docker is an **open-source** platform.
- ✓ It packages an application and all its dependencies into a **container**.
- ✓ Runs **consistently** on any environment.
- ✓ Lightweight, portable and easy to deploy.
- ✓ Widely used in **DevOps** & **CI/CD** pipelines.

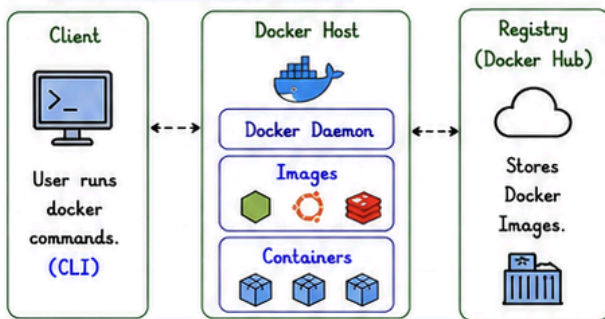


★ 2. HOW DOCKER WORKS?



Client sends commands → Daemon manages
Images → Containers are created and run.

★ 3. DOCKER ARCHITECTURE



★ 4. KEY DOCKER CONCEPTS

Concept	Description
Image	A read-only template with instructions to create a container.
Container	A running instance of an image.
Dockerfile	A text file with instructions to build a Docker image.
Docker Hub	A cloud registry to store & share Docker images.
Volume	Used to persist data outside container.
Network	Allows containers to communicate with each other.

★ 5. DOCKER BASIC COMMANDS

```

docker --version      # Check Docker version
docker pull <image>   # Download an image
docker images         # List all images
docker run <image>    # Run a container
docker ps             # List running containers
docker ps -a          # List all containers
docker stop <id>      # Stop a container
docker rm <id>        # Remove a container

```

★ 6. DOCKERFILE EXAMPLE

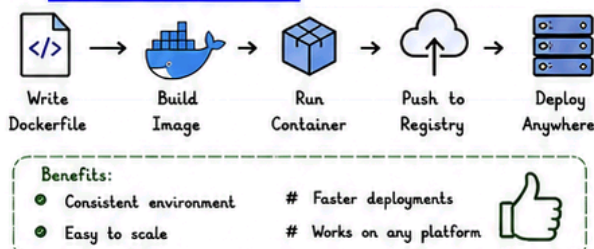
```

FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

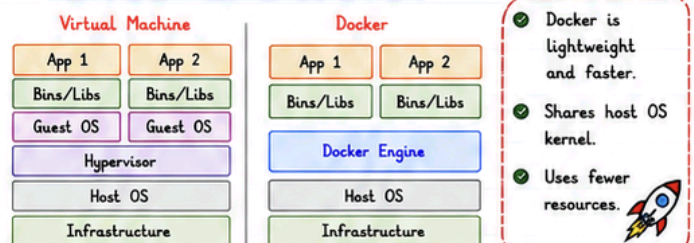
```

← Base image
 ← Set working directory
 ← Copy jar file
 ← Expose port
 ← Run the application

★ 7. DOCKER WORKFLOW



★ 8. DOCKER VS VIRTUAL MACHINE



KEY TAKEAWAY

Docker makes applications **portable**, **consistent** and **easy to deploy**.
It is a core technology in **DevOps** and **Cloud Native** world.





KUBERNETES

Kubernetes (K8s) is an **open-source** container orchestration platform that automates **deployment**, **scaling** and management of **containerized** applications.

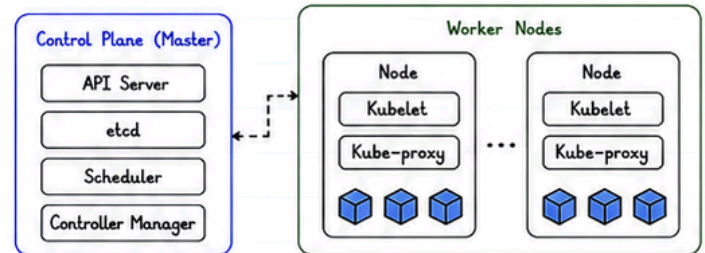
★ 1. WHAT IS KUBERNETES?

- ✔ Kubernetes **automates** deploying, scaling and operating containerized applications.
- ✔ It groups containers into **logical units** for easy management.
- ✔ Provides **self-healing**, load balancing and rolling updates.
- ✔ Widely used in **DevOps** and **Cloud Native** environments.



kubernetes

★ 2. KUBERNETES ARCHITECTURE



Control Plane → Manages the cluster

Worker Nodes → Run the containerized applications

Pods → Smallest deployable units (one or more containers)

★ 3. KUBERNETES CORE CONCEPTS

Pod	Deployment	Service	Namespace	ConfigMap
Smallest unit that can be deployed.	Manages ReplicaSets and provides declarative updates.	Stable endpoint to access a set of Pods.	Virtual cluster inside a physical cluster.	Store non-sensitive configuration data in key-value pairs.

★ 4. KUBERNETES OBJECTS

Object	Purpose
Pod	Runs one or more containers.
Deployment	Ensures desired number of pod replicas are running.
Service	Exposes pods to other pods or outside the cluster.
ConfigMap	Stores configuration data as key-value pairs.
Secret	Stores sensitive information like passwords, tokens.
Ingress	Manages external access to services (HTTP/HTTPS).

★ 5. KUBERNETES BASIC COMMANDS

```

kubectl get pods           # List all pods
kubectl get nodes          # List all nodes
kubectl get deployments    # List all deployments
kubectl get services       # List all services
kubectl describe pod <pod-name> # Describe a pod
kubectl logs <pod-name>    # View pod logs
kubectl apply -f <file.yaml> # Apply config
kubectl delete -f <file.yaml> # Delete config
  
```

★ 6. POD YAML EXAMPLE

```

apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80
  
```

← K8s API version

← Object type

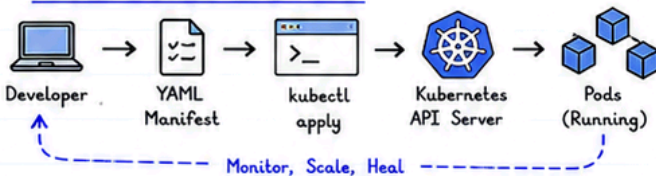
← Pod name

← Container name

← Container image

← Exposed port

★ 7. KUBERNETES WORKFLOW



★ 8. BENEFITS OF KUBERNETES

- ✔ Automated deployment and scaling
- ✔ High availability & self-healing
- ✔ Load balancing
- ✔ Efficient resource usage
- ✔ Rolling updates & rollback
- ✔ Portable across clouds and on-premises



KEY TAKEAWAY

Kubernetes **automates** container management, makes applications **scalable**, **reliable** and **easier** to operate in modern IT environments.





MONITORING & LOGGING

Monitoring helps you know **what is happening** in your systems in real-time.

Logging helps you understand **why** it happened.

★ 1. WHAT IS MONITORING?

- Monitoring is collecting **real-time metrics** from systems and services.
- It helps in **detecting issues early** and ensuring system health.
- Focuses on **metrics, performance** and **alerts**.



★ 2. WHAT IS LOGGING?

- Logging is recording **events** and logs from systems.
- It helps in **troubleshooting** and root cause analysis.
- Focuses on **logs, events** and **details**.

Log File

```
2024-05-20 10:15:21 INFO User login success
2024-05-20 10:15:22 ERROR DB connection failed
2024-05-20 10:15:23 WARN High memory usage
...
```

★ 3. MONITORING ARCHITECTURE

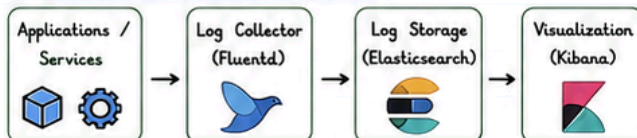


- Exporter** collects metrics from services.
- Metrics are stored in **Time Series DB (Prometheus)**.
- Grafana** visualizes data and sends **alerts**.

★ 4. POPULAR MONITORING TOOLS

Tool	Purpose
Prometheus	Collects & stores metrics, powerful querying.
Grafana	Visualizes metrics with dashboards.
Alertmanager	Handles alerts from Prometheus.
Node Exporter	Exports system metrics (CPU, Memory, Disk).
Loki	Log aggregation system by Grafana Labs.
cAdvisor	Monitors containers performance.

★ 5. LOGGING ARCHITECTURE



- Collector** collects logs from multiple sources.
- Logs are stored in **Elasticsearch**.
- Kibana** helps search & visualize logs.

★ 6. POPULAR LOGGING TOOLS (ELK STACK)

Tool	Purpose
Elasticsearch	Stores and indexes logs.
Logstash / Fluentd	Collects, parses and sends logs.
Kibana	Searches and visualizes logs.

★ 7. SAMPLE PROMETHEUS METRIC

```
# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",status="200"} 1523
http_requests_total{method="POST",status="500"} 23
http_request_duration_seconds_bucket{le="0.1"} 120
http_request_duration_seconds_bucket{le="0.5"} 340
...
```

← Metric name
← Labels
← Values
← Histogram example

★ 8. SAMPLE LOG ENTRY (JSON)

```
{
  "timestamp": "2024-05-20T10:15:21Z",
  "level": "ERROR",
  "service": "user-service",
  "message": "DB connection failed",
  "trace_id": "abc123xyz",
  "user_id": "1001"
}
```

← When it happened
← Severity level
← Which service
← What happened
← Trace ID
← Additional info

★ BONUS: MONITORING vs LOGGING

Aspect	MONITORING	LOGGING
Purpose	Tells you WHAT is happening (in real-time)	Tells you WHY it happened
Data	Metrics (numbers, counts, rates)	Logs (events, messages, details)
Tools	Prometheus, Grafana, Alertmanager	ELK Stack, Loki, Fluentd
Use Case	Alerts, dashboards, system health	Troubleshooting, audit, debugging
Data Type	Numeric	Textual

BEST PRACTICES

- Monitor everything, alert on anything.
- Centralize logs for easy search.
- Keep dashboards simple & useful.
- Set meaningful alerts, avoid noise.
- Use retention & compression for logs.



KEY TAKEAWAY

Monitoring shows you the health, **Logging** gives you the details.
Together, they make your systems reliable!



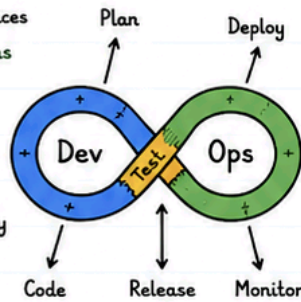


DEVOPS IN AUTOMATION TESTING

DevOps and Test Automation together help deliver **high-quality** software **faster**, more **reliably** and **continuously**.

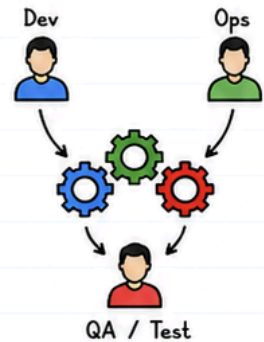
★ 1. WHAT IS DEVOPS IN TESTING?

- DevOps is a culture + set of practices that bring **Development**, **Operations** and **Testing** teams together.
- Involves collaboration, **automation** and continuous improvement.
- Goal: Deliver value to users quickly with **high quality**.

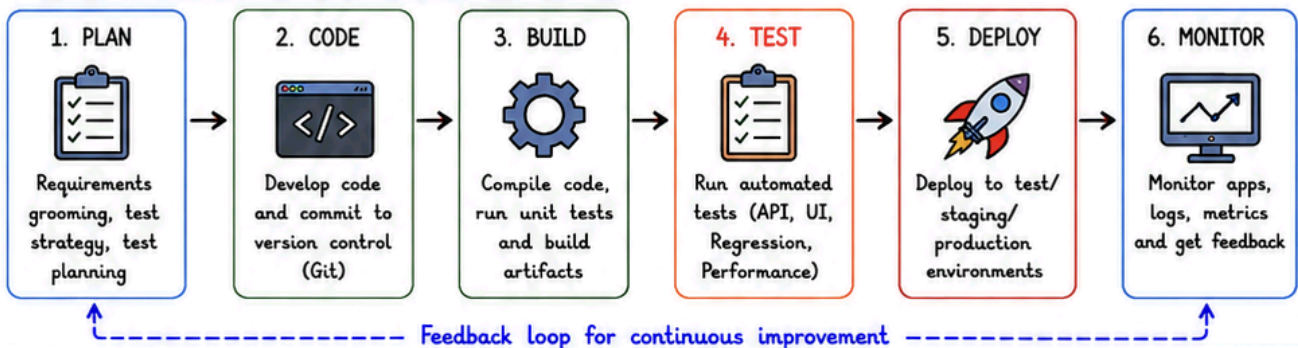


★ 2. HOW DEVOPS HELPS AUTOMATION TESTING?

- Early **bug detection**
- Faster **feedback**
- Continuous **testing**
- Quick** releases
- Better collaboration
- High product **quality**



★ 3. DEVOPS PIPELINE WITH AUTOMATION TESTING



★ 4. TOOLS USED

Stage	Tools (Examples)
Code	Git, GitHub, GitLab, Bitbucket
Build	Jenkins, GitLab CI, Maven, Gradle
Test Automation	Selenium, Playwright, Cypress, Postman, Rest Assured
Test Management	TestNG, Cucumber, JUnit, Jira, Zephyr
Deploy	Docker, Kubernetes, Helm
Monitoring	Prometheus, Grafana, ELK Stack

★ 5. BEST PRACTICES

- Shift-left testing (test **early**).
- Automate **repeatable** tests.
- Maintain test data & environment **as code**.
- Integrate tests in **CI/CD** pipeline.
- Monitor** results and act quickly.
- Collaborate across Dev, QA and Ops.

Automate everything that can be automated, but **intelligently**!

★ 6. EXAMPLE: JENKINS PIPELINE (SAMPLE)

```
pipeline {
  agent any
  stages {
    stage('Build') { steps { sh 'mvn clean install' } }
    stage('Test') { steps { sh 'mvn test' } }
    stage('Deploy') { steps { sh 'docker build .' } }
  }
}
```

- ← Run on any available agent
- ← Build the project
- ← Run automated tests
- ← Build Docker image (deploy step)

★ 7. KEY BENEFITS



KEY TAKEAWAY

When **DevOps** and **Test Automation** work together, you get faster releases, **happier users** and **reliable software**!

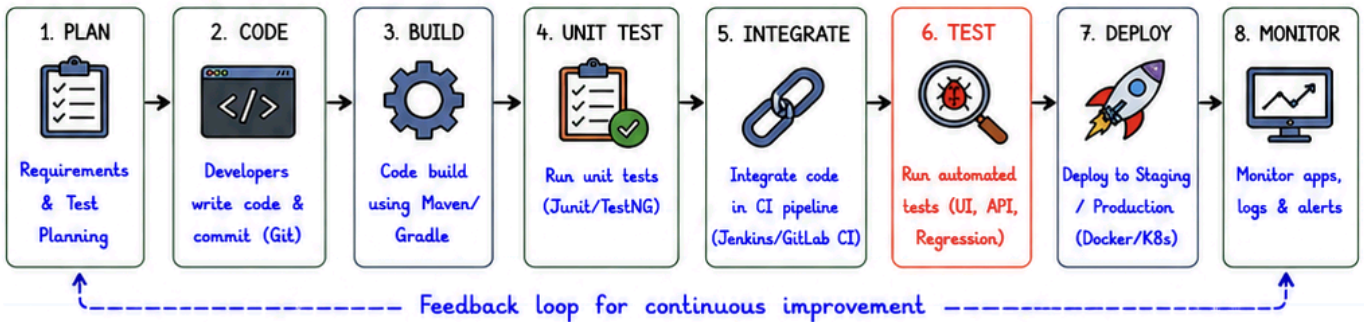




REAL PROJECT WORKFLOW (END TO END)

Let's see how a real project flows using DevOps and Automation Testing from **code commit** to **production** with **monitoring**.

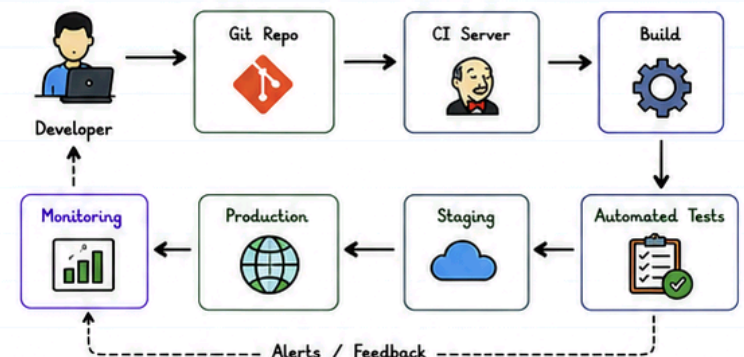
★ 1. END TO END WORKFLOW OVERVIEW



★ 2. DETAILED FLOW (STEP BY STEP)

- Developer writes code and pushes to **Git repository**.
- CI server triggers **build** automatically.
- Unit tests run - if **failed**, pipeline stops.
- Code is integrated and **artifact** is created.
- Automated tests** (UI/API) execute in **test environment**.
- If all tests pass, code is deployed to **staging**.
- After validation, code is promoted to **production**.
- Application is monitored for performance, errors & logs.

★ 3. REAL PROJECT WORKFLOW DIAGRAM



★ 4. SAMPLE CI PIPELINE (JENKINSFILE)

```
pipeline {
  agent any
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build') { steps { sh 'mvn clean install' } }
    stage('Unit Test') { steps { sh 'mvn test' } }
    stage('Deploy to Staging') { steps { sh 'deploy.sh staging' } }
    stage('UI Tests') { steps { sh 'mvn test -Dtest=UITest' } }
    stage('Deploy to Prod') { steps { sh 'deploy.sh prod' } }
  }
}
```

← Get code from repo

← Build the project

← Run unit tests

← Deploy to staging

← Run UI automation tests

← Deploy to production

★ 5. TOOLS USED IN REAL WORKFLOW

Stage	Tools / Technologies
Code	Git, GitHub, GitLab, Bitbucket
Build	Maven, Gradle
CI/CD	Jenkins, GitLab CI, GitHub Actions
Test Automation	Selenium, Playwright, Rest Assured, TestNG, JUnit
Deployment	Docker, Kubernetes, Helm
Monitoring	Prometheus, Grafana, ELK Stack

★ 6. KEY TAKEAWAYS

- DevOps enables **collaboration**, **automation** and faster delivery.
- Automation Testing ensures **quality** at every stage.
- CI/CD pipeline gives consistent and **reliable** releases.
- Monitoring helps in quick **issue detection** and resolution.
- Continuous feedback improves the product **continuously**.



★ 7. REAL WORLD SCENARIO

Whenever a **developer** pushes code, the entire pipeline runs automatically. If everything passes, users get the new feature without manual effort and with **high quality**.



KEY TAKEAWAY

A good **DevOps workflow** + strong **automation** = **Reliable Releases**, **Happy Teams** and **Happy Customers**!





BEST PRACTICES & INTERVIEW TIPS

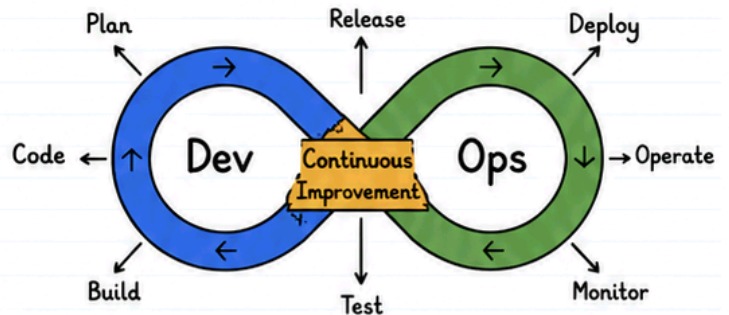


Follow best practices to make your DevOps pipeline **reliable**, **secure** and **efficient**.

These tips also help you **crack** DevOps & Automation Testing interviews!

★ 1. BEST PRACTICES

- ✔ Automate everything - Build, Test, Deploy, Monitor.
- ✔ Keep code in **version control** (Git).
- ✔ Use **small & frequent** commits.
- ✔ Run **tests early** and often.
- ✔ Keep environments **consistent**.
- ✔ **Monitor** everything & act on alerts.
- ✔ Use **Infrastructure as Code (IaC)**.
- ✔ Keep pipelines **simple, secure & repeatable**.



★ 2. PIPELINE BEST PRACTICES



★ 3. CODE QUALITY TIPS

- ✔ Follow coding **standards**.
- ✔ Do **code reviews**.
- ✔ Use **linters & static code analysis** (SonarQube, ESLint).
- ✔ Write meaningful **commit messages**.
- ✔ Remove **duplicate** code.



★ 4. SECURITY BEST PRACTICES



- ✔ Never **hardcode secrets**.
- ✔ Use **environment variables & credential stores**.
- ✔ Keep dependencies **updated**.
- ✔ Scan code for **vulnerabilities**.
- ✔ Apply **least privilege** access.

★ 7. DEVOPS SUCCESS FORMULA



★ 5. INTERVIEW TIPS

- 💡 **Understand Concepts:**
Focus on "Why" not just "How".
- 🎯 **Be Hands-on:**
Practice building real pipelines.
- 💬 **Explain Clearly:**
Use simple diagrams to explain.
- 📖 **Know Tools:**
Git, Jenkins, Docker, Kubernetes, SonarQube, Prometheus.
- 👤 **Share Experience:**
Talk about projects, challenges & how you solved them.

★ 6. COMMON INTERVIEW QUESTIONS

- Q1. What is CI/CD?
Ans: CI/CD is a practice of automating code integration (CI) and delivery/deployment (CD).
- Q2. Difference between Docker & Kubernetes?
Ans: Docker is for containerization. Kubernetes is for orchestration.
- Q3. What is the role of Jenkins in DevOps?
Ans: Jenkins automates build, test and deployment using pipelines.
- Q4. How do you handle secrets in Jenkins?
Ans: Use Jenkins Credentials or external secrets managers.

★ 8. QUICK REMEMBER

C - Code
I - Integrate
T - Test
D - Deploy
O - Operate
M - Monitor

} CI/CD + Monitoring
= Reliable Delivery



KEY TAKEAWAY

Follow best practices, keep learning and practice **real tools**.

That's how you become a **DevOps + Automation Testing Expert!** 🚀





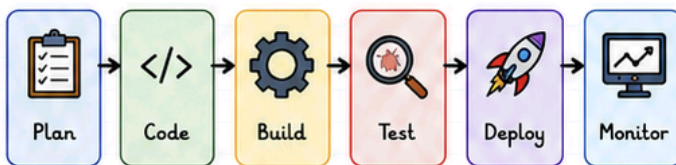
SUMMARY & QUICK REVISION



Let's quickly revise everything we learned in this DevOps Series.

Keep learning, keep **building**, keep **automating**! 🚀

★ 1. DEVOPS LIFECYCLE (RECAP)



Continuous **Integration**, Continuous Delivery and Continuous **Feedback** make it repeatable and reliable.

★ 2. DEVOPS BENEFITS

- ✓ **Faster** time to market
- ✓ High quality software
- ✓ Early **issue detection**
- ✓ Better collaboration
- ✓ Continuous improvement
- ✓ Happy teams & customers



People + Process + Tools = **Success**

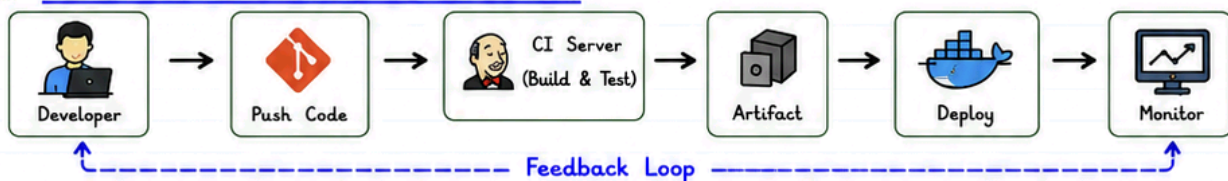
★ 3. KEY CONCEPTS WE LEARNED

- ✓ **CI** - Integrate code frequently & automatically.
- ✓ **CD** - Deliver changes to production safely.
- ✓ **Automation** - Saves time, reduces manual effort.
- ✓ **Monitoring** - Get visibility & act on insights.
- ✓ **Collaboration** - Dev + Ops + QA work together.

★ 4. TOOLS QUICK RECAP

Area	Tools
Code	Git, GitHub, GitLab, Bitbucket
Build	Maven, Gradle
CI/CD	Jenkins, GitLab CI, GitHub Actions
Test Automation	Selenium, Playwright, Rest Assured, TestNG, JUnit
Deployment	Docker, Kubernetes, Helm
Monitoring	Prometheus, Grafana, ELK Stack

★ 5. REAL PROJECT WORKFLOW (SHORT RECAP)



★ 6. DEVOPS + AUTOMATION TESTING

- ✓ Automated tests run in **CI pipeline**.
- ✓ Fast **feedback** to developers.
- ✓ **Quality gates** ensure only good code moves forward.
- ✓ Test **reports & metrics** help in decision making.
- ✓ **Shift-left** testing improves quality early.



★ 7. INTERVIEW QUICK TIPS

- ✓ Understand concepts clearly.
- ✓ Give **real examples** from projects.
- ✓ Use simple **diagrams**.
- ✓ Know your **tools & commands**.
- ✓ Practice explaining **end to end flow**.



Example Question

Q. How does CI/CD pipeline work?

Ans: Code commit → Build → Test → Deploy → Monitor → Feedback.

★ 8. IMPORTANT COMMANDS (QUICK REVISION)

```

git clone <repo_url> # Clone repository
git add . # Stage all changes
git commit -m "message" # Commit changes
git push origin main # Push to remote
docker build -t app . # Build Docker image
docker run -d -p 8080:8080 app # Run container
  
```

★ 9. ONE LINE TAKEAWAY



DevOps brings **People**, **Process** and **Tools** together and **Automation** makes it **FASTER**, **SMARTER** and **RELIABLE**. ✓



You've completed the **DevOps Series**! 🎉
Keep practicing, keep **building**, keep **growing**! 🚀

